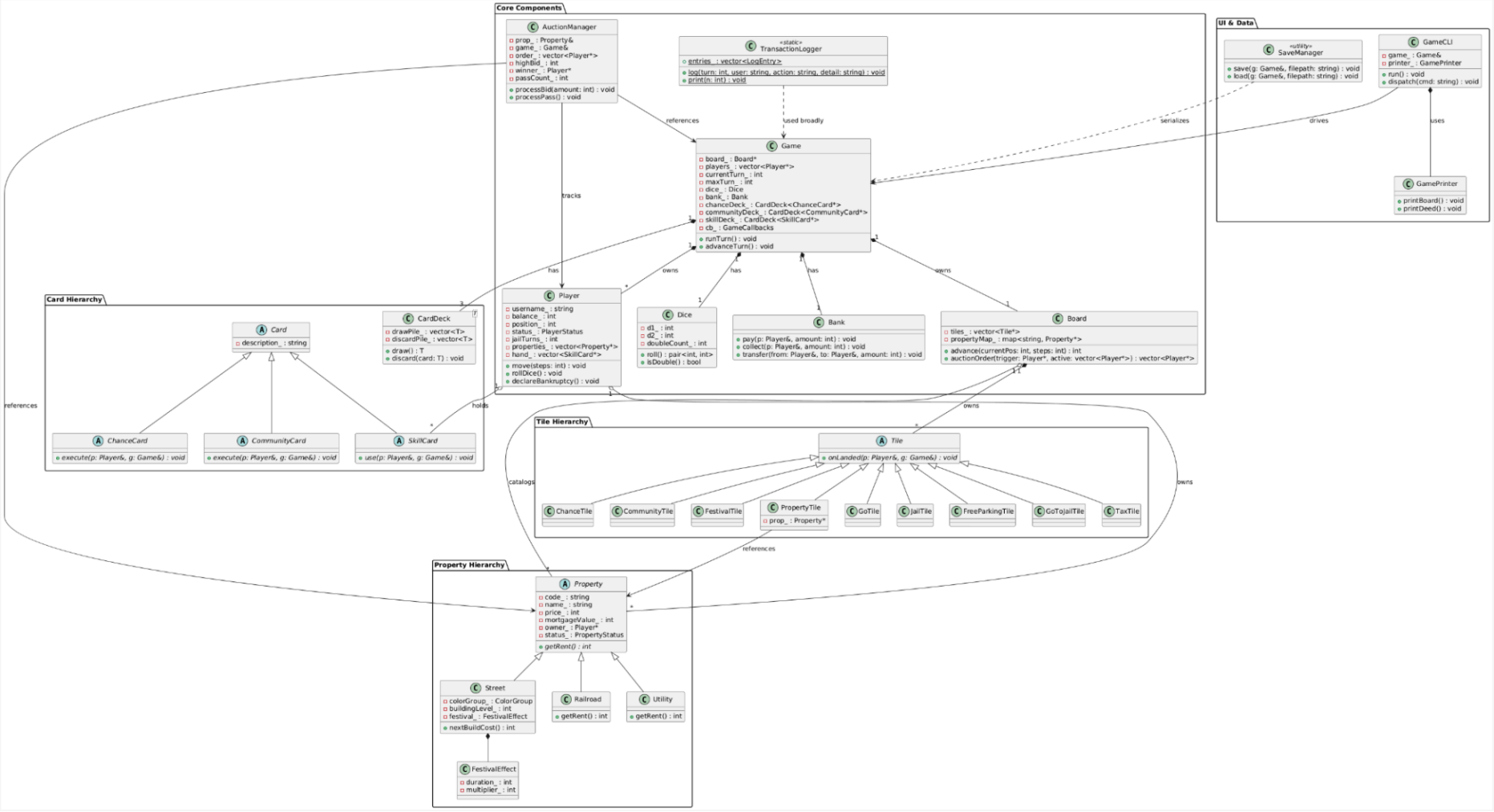
Laporan M2 Tugas Besar 1
IF2010 Pemrograman Berorientasi Objek 2025/2026
Nimonspoli

Kode Kelompok : OMB
Nama Kelompok : OOPs my bad

1. 13524009 / Mikhael Benrael Tampubolon
2. 13524021 / Natanael I. Manurung
3. 13524085 / Ariel Cornelius Sitorus
4. 13524107 / Rava Khoman Tuah Saragih
5. 13622076 / Ziyan Agil Nur Ramadhan

OMB - OOPs my bad

1. Diagram Kelas



Gambar 1.1 Diagram Kelas Nimonspoli

2. Deskripsi Kelas

2.1. Kelas Game

Kelas Game adalah kelas utama (Sebisa mungkin tanpa menjadi god class) dan wadah status (State Container) untuk aplikasi ini. Tanggung jawab utamanya adalah mengatur alur permainan tingkat tinggi (seperti pergantian giliran dan mengecek kondisi kemenangan), namun secara spesifik mendelegasikan eksekusi mekanik ke entitas bawahannya (seperti Player atau Board).

Pengaturan State Global: Kelas ini memiliki kendali eksklusif terhadap instance Board, daftar Player, konfigurasi khusus (melalui SpecialConfig dan TaxConfig), utilitas Bank dan Dice, serta seluruh tumpukan dek kartu (chanceDeck_, communityDeck_, skillDeck_). Game memastikan bahwa semua state ini terhubung satu sama lain.

Sistem Callback Antarmuka (UI): Untuk menghindari ketergantungan pada pustaka input/output standar (cin/cout), Game menyimpan atribut class GameCallbacks cb_. Struct ini berisi fungsi anonim (lambda) yang diinjeksikan oleh antarmuka (contohnya GameCLI). Ketika sebuah keputusan dari pengguna dibutuhkan di tengah pemrosesan logika (misalnya konfirmasi untuk membeli properti, atau input target lelang), Game hanya akan memanggil callback tersebut dan menunggu return value tanpa perlu menangani prompt secara mandiri

Siklus Pemain (Turn Cycle): Fungsi utama runTurn() akan memanggil pemain berdasarkan urutan di turnOrder_ dan memantau perpindahan dari satu pemain ke pemain lain melalui fungsi advanceTurn(). Jika ada efek permainan yang berbasis giliran (misalnya sisa waktu FestivalEffect pada sebuah jalan), Game akan mengurangi durasi tersebut pada saat giliran berpindah. Secara aktif pula, kelas ini menjalankan fungsi checkWinCondition() yang akan menghentikan permainan jika hanya tersisa satu pemain yang aktif (tidak bangkrut) atau jumlah giliran maksimal telah terlampaui.

2.2. Kelas Player

Kelas Player memodelkan entitas manusia di dalam logika mesin permainan. Entitas ini mewakili segala hal yang "dimiliki" dan "dilakukan" oleh pengguna.

Manajemen Inventory: Pemain menyimpan dua hal utama di dalam kantongnya: kumpulan pointer ke kepemilikan aset (properties_) dan kumpulan kartu kemampuan/SkillCard (hand_). Khusus untuk kartu kemampuan, Player memiliki proteksi maksimal isi tangan (MAX_HAND_SIZE) dan akan menolak penambahan jika limit telah tercapai, memaksa intervensi antarmuka (drop card).

Evaluasi Keuangan (Net Worth vs. Liquidation): Kelas ini tidak hanya menyimpan uang tunai (balance_), tetapi bertanggung jawab memberikan kalkulasi finansial tingkat lanjut. Fungsi netWorth() digunakan untuk menghitung pajak PPH persentase (uang tunai + harga beli rumah/hotel + harga tanah murni). Sebaliknya, fungsi maxLiquidation() dihitung berdasarkan uang tunai ditambah harga jual agunan (gadai) properti jika seluruh aset dilikuidasi ke Bank (digunakan saat mengecek potensi kebangkrutan untuk hutang).

Pendelegasian Logika Mekanis: Player mengatur fungsi-fungsi geraknya secara mandiri. Saat rollDice() dipanggil, pemain akan memeriksa apakah ia dipenjara. Jika tidak, ia akan mengocok Dice, merekam transaksi, mencatat pelemparan (flag hasRolled_), dan langsung berpindah koordinat. Jika ia menggulir double hingga 3 kali, ia men-trigger goToJail(). Fungsi handleJailTurn() mendikte logika pembebasan. Pada percobaan penjara ke-1 dan ke-2, pemain dipaksa bertaruh dengan double. Jika gagal, jailTurns_ naik. Jika masuk giliran ke-3, pemain diwajibkan membayar denda untuk keluar, atau dipaksa menyatakan kebangkrutan jika tak mampu membayar.

Sistem Bangkrut (Bankruptcy): Saat pemain tak mampu memenuhi hutang bahkan setelah melikuidasi seluruh aset, fungsi declareBankruptcy() akan dipicu. Jika berhutang pada pemain lain, maka semua uang sisa dan sertifikat properti berpindah nama secara otomatis ke kreditor tersebut. Namun, jika ia bangkrut karena sistem (seperti pajak atau dokter), bangunan di jalan tersebut dihancurkan tanpa kompensasi, dan tanah kosongan dikembalikan ke Bank untuk selanjutnya dilelang dari nol.

2.3. Kelas Board

Kelas ini adalah infrastruktur yang memegang tanggung jawab penuh terhadap bentuk fisik papan permainan, pengelolaan matriks petak, dan penyusunan aset properti.

Penyimpanan Matriks RAII: Menyimpan urutan petak menggunakan std::vector<std::unique_ptr<Tile>>. Pendekatan *Resource Acquisition Is Initialization* (RAII) ini memastikan manajemen memori yang aman; saat objek hancur, seluruh *pointer* petak ikut terhapus otomatis.

Sistem Rotasi Pola Melingkar: Memfasilitasi pergerakan *looping* menggunakan *modulo mathematics*: (currentPos + steps) % tiles_.size(). Transisi nilai indeks dari besar ke kecil menjadi detektor logis bahwa pemain telah melintasi garis START/GO.
Katalog Akses Cepat: Memelihara struktur *hash map* (std::map<string, Property*>) untuk memetakan kode petak langsung ke *pointer* propertinya. Hal ini memangkas inefisiensi kompleksitas waktu menjadi O(1) untuk pencarian properti saat lelang atau pembangunan.

Logika Antrean Lelang: Menghitung auctionOrder() untuk mengatur ulang daftar giliran pemain yang berhak menawar pertama kali secara adil dan dinamis berdasarkan posisi pemain saat itu.

2.4. Kelas Bank

Kelas ini adalah fasilitator transaksi finansial independen yang merepresentasikan sistem ekonomi pusat tanpa batas simpanan uang.

Abstraksi Keuangan Bebas: Mengelola uang beredar melalui tiga metode sentral. Metode pay() mencetak uang baru (inflasi) untuk pemain, collect() menarik uang dari peredaran (deflasi) untuk pembayaran pajak/denda, dan transfer() menjembatani perpindahan uang antar pemain tanpa memengaruhi total peredaran uang sistem.

2.5. Kelas Dice

Kelas ini adalah representasi mekanik pengacakan ganda untuk dadu dalam permainan.

Mekanik Mersenne Twister: Menggunakan generator pseudo-acak std::mt19937 untuk menjamin distribusi angka 1-6 yang adil dan berimbang.

Manajemen State Kembar: Melacak hasil pelemparan ganda (d1_ == d2_) dan menyimpan hitungan *double*. State ini dapat diakses oleh kelas lain (terutama Player) untuk mengevaluasi bonus giliran atau penalti penjara.

Injeksi Eksplisit untuk Pengujian: Memiliki fungsi setRoll(d1, d2) yang memotong jalur (*bypass*) mesin pengacakan, sangat berguna untuk pengujian logika (testing) melalui antarmuka *command-line*.

2.6. Kelas Auction Manager

Kelas ini adalah *state machine* sementara (*ephemeral*) yang hidup eksklusif hanya untuk memproses satu siklus transaksi lelang secara bergiliran.

Navigasi Lelang (Tracking): Menyimpan kandidat pemenang sementara (winner_) dan rekor tawaran tertinggi (highBid_). Setiap kali pemain memasukkan angka pada fungsi processBid(), sistem memperbarui rekor dan mereset penghitung menyerah (*pass count*). Jika pemain menyerah, processPass() dipanggil.

Terminasi State: Lelang dianggap selesai (isFinished()) jika jumlah pemain yang menyerah sama dengan sisa peserta dan terdapat pemenang yang valid. Jika seluruh pemain menyerah di putaran pertama tanpa ada yang menawar, sistem merespons isFailed() yang akan ditangkap UI untuk memaksa perulangan agar minimal ada penawaran awal.

2.7. Kelas TrancactionLogger

Kelas ini adalah auditor pencatatan (Log) berskala global untuk merekam setiap jejak transaksi dan aksi secara persisten.

Desain Objek Statis Global: Diimplementasikan dengan kontainer statis tunggal (static std::vector<LogEntry>). Pola ini memudahkan pemanggilan *logging* secara instan dari hierarki kode terdalam tanpa harus melempar instansi kelas ke setiap fungsi.

Pemformatan UI Terminal: Berfungsi juga sebagai pelayan UI melalui metode print(n) yang dapat merangkai parameter log, menambahkan bingkai estetis terminal, dan menyortir jumlah keluaran sesuai parameter ukuran riwayat yang diminta.

2.8. Kelas Tile (Abstrak)

Kelas ini adalah kontrak antarmuka (*Interface*) utama bagi setiap petak koordinat di atas papan permainan.

Pola Desain Command/Visitor: Mewajibkan adanya implementasi metode virtual onLanded(Player&, Game&). Desain ini krusial karena Game tidak perlu menulis rantai pernyataan *if-else* masif untuk mendeteksi petak. Game cukup memanggil metode tersebut, dan petak yang bersangkutan akan mengeksekusi logikanya sendiri.

2.8.1 Kelas GoTile

Kelas ini adalah implementasi petak konkret untuk posisi GO/Start.

Pemicu Hadiah: Saat mendarat atau terlewati, kelas ini memanggil fungsi Bank untuk mendistribusikan hadiah injeksi uang gaji (*salary*) kepada pemain bersangkutan.

2.8.2 Kelas JailTile & FreeParkingTile

Kelas ini adalah implementasi petak konkret untuk area transit aman.

Petak Pasif: Pemain dapat singgah di petak ini tanpa menerima sanksi finansial, pajak, maupun hambatan pergerakan di giliran berikutnya.

2.8.3 Kelas GoToJailTile

Kelas ini adalah implementasi petak konkret yang bersifat represif secara hukum.

Relokasi Paksa: Secara paksa mengeksekusi prosedur goToJail() pada pemain yang mendarat, mencabut hak jalan ekstra, dan langsung merelokasi *pointer* posisi ke dalam penjara seketika.

2.8.4 Kelas TaxTile

Kelas ini adalah implementasi petak konkret untuk retribusi pajak wilayah.

Pajak Dinamis: Berinteraksi langsung dengan saldo pemain untuk menarik tarif. Jika jenis petak memberlakukan aturan Pajak Penghasilan (PPh), kelas ini akan memicu *callback* antarmuka agar pengguna dapat memilih kalkulasi denda (flat tetap atau potongan persentase dari aset properti).

2.8.5 Kelas ChanceTile & CommunityTile

Kelas ini adalah implementasi petak konkret penyedia probabilitas efek kartu.

Pemicu Dek Kartu: Memerintahkan kelas inti Game untuk mencabut (draw) kartu teratas dari CardDeck yang berkorespondensi, lalu langsung memanggil eksekusi efek kartu terhadap pemain tersebut

.

2.8.6 Kelas FestivalTile

Kelas ini adalah implementasi petak konkret yang memberikan bonus perkalian wilayah.

Pemberian Berkah Wilayah: Saat diinjak, petak ini menghentikan sementara siklus otomatis perpindahan pemain dan mendesak pemain memilih satu petak Street miliknya melalui UI untuk diberikan FestivalEffect.

2.8.7 Kelas PropertyTile

Kelas ini adalah petak fisik di matriks papan yang mereferensikan aset properti nyata.

Jembatan Ekonomi-Papan: Menyambungkan keberadaan linier koordinat pemain dengan objek ekonomi abstrak Property. Petak ini menangani logika deteksi kepemilikan. Jika properti milik saingan, uang sewa ditarik; jika kosong, akan langsung ditawarkan ke pemain, atau dilempar ke AuctionManager jika ditolak.

2.9. Kelas Property

Kelas ini adalah fondasi hierarki yang mendefinisikan seluruh perilaku ekonomi dari suatu aset kepemilikan di dalam permainan.

Penyimpanan State Ekonomi: Menyimpan properti absolut seperti harga beli, nilai gadai, *pointer* pemilik saat ini (owner_), dan status ekonomi (MORTGAGED, OWNED, BANK).

Kontrak Polimorfisme Sewa & Gadai: Menyediakan *pure virtual method* agar kalkulasi harga sewa dieksekusi sesuai spesifikasi sub-kelas. Menstandarisasi penanganan perpindahan kepemilikan saat lelang usai, prosedur pegadaian, dan pelunasan gadai.

2.9.1 Kelas Street

Kelas ini adalah sub-kelas Property komersial yang dapat dibangun dan menjadi poros rotasi ekonomi permainan.

Skala Pembangunan Rumah: Memonitor elevasi konstruksi (level 0 tanah hingga level 5 hotel). Setiap tingkat menyesuaikan biaya nextBuildCost().

Pengali Color Group: Terikat dengan skema zonasi grup warna. Pendudukan dominan oleh satu entitas memicu skema "Monopoli" yang seketika melipatgandakan sewa dasar meskipun belum ada bangunan fisik di atasnya.

Integrasi Efek Festival: Menjadi satu-satunya jenis properti yang menampung objek FestivalEffect, yang mana jika dipicu akan memberikan faktor pengali ekstrem (x2 hingga x8) pada sewa.

2.9.2 Kelas Railroad

Kelas ini adalah sub-kelas Property bertemakan moda transportasi publik.

Dinamika Sewa Berjenjang: Tidak bergantung pada konstruksi bangunan. Fungsi penghitungan sewanya secara cerdas (*on-the-fly*) memindai portofolio pemilik, menghitung kepemilikan stasiun lain, dan mengeksponensiasi harga sewa dasar berdasarkan jumlah monopoli stasiun.

2.9.3 Kelas Utility

Kelas ini adalah sub-kelas Property bertemakan layanan kelistrikan dan perairan umum.

Tarif Volatil Berbasis RNG: Nilai penagihan sewanya murni merujuk pada lastDiceTotal() (angka dadu terakhir pemain yang mendarat) lalu mengalikannya dengan faktor kepemilikan utilitas (pengali 4 atau 10), menghasilkan tagihan sewa yang sangat acak tiap putarannya.

2.10. Kelas Festival Effect

Kelas ini adalah komponen utilitas ringan yang ditanamkan (*composed*) langsung ke dalam Street.

Manajemen Daur Waktu: Menghitung *countdown* efek. Sistem ini dikaitkan dengan alur turn utama Game untuk menurunkan durasinya satu per satu seiring berjalannya putaran.

Amplifikasi Nilai Sewa: Menyimpan jumlah *multiplier*. Jika efek festival ditimpa pada petak yang sama, kelas ini secara aman mereset durasi waktu atau mengeksponensiasi pengganda harga maksimum sesuai konfigurasi.

2.11. Kelas Card (Abstrak)

Kelas ini adalah polimorfik dasar (*Base Class*) yang jadi standar fungsi seluruh varian kartu permainan.

Antarmuka Eksekusi Bebas: Menetapkan kontrak *virtual method* execute() atau use(), yang mana implementasi teknisnya (seperti memotong uang atau merelokasi matriks) sepenuhnya dilepaskan pada imajinasi sub-kelas masing-masing

.

2.11.1 Kelas ChanceCard & CommunityCard

Kelas ini adalah sekumpulan kartu pasif-reaktif yang memicu intervensi mendadak saat dicabut.

Intervensi Navigasi Lintas Petak: Dapat membajak *pointer* posisi (misal: Mundur 3 Langkah, atau Pindah Ke Stasiun Terdekat) dan secara instan memanggil event mendarat (onLanded) pada lokasi target.

Manipulasi Moneter Antar-Pemain: (Khusus *Community*). Menjalankan perulangan finansial secara sekuensial yang merugikan atau menguntungkan satu pemain berbanding populasi massa yang tersisa (misal: Biaya Ulang Tahun atau Skandal Pemilu).

2.11.2 Kelas SkillCard (Abstrak)

Kelas ini adalah kategori kartu spesial bersistem inventaris yang proaktif, di mana inisiasi penggunaannya diserahkan sepenuhnya kepada *user* (melalui antarmuka).

2.11.3 Kelas MoveCard

Kelas ini adalah sub-kelas SkillCard manipulasi posisi pergerakan.

Navigasi Modular: Memodifikasi posisi bidak pemain tanpa mengocok dadu, berguna untuk mengejar properti target atau lari dari area bahaya tanpa memicu penalti ganda dadu.

2.11.4 Kelas DiscountCard

Kelas ini adalah sub-kelas SkillCard pendukung finansial.

Subsidi Pembelian/Pembayaran: Digunakan sebagai *buffer* perhitungan biaya untuk mendapatkan potongan harga mutlak (diskon) saat melakukan transaksi pembangunan rumah atau melunasi tagihan denda.

2.11.5 Kelas ShieldCard

Kelas ini adalah sub-kelas SkillCard untuk proteksi darurat.

Tameng Kebal Penagihan: Kartu reaktif yang ditahan di saku pemain, dapat dibakar (*consumed*) secara instan tepat ketika pemain diwajibkan membayar sewa mahal yang berpotensi menyebabkan kebangkrutan.

2.11.6 Kelas TeleportCard

Kelas ini adalah sub-kelas SkillCard transportasi presisi tingkat tinggi.

Relokasi Bebas Hambatan: Menerima *input* koordinat spesifik dari UI, memungkinkan pemain menunjuk indeks petak absolut tanpa peduli di mana posisi mereka saat ini berlabuh.

2.11.7 Kelas LassoCard

Kelas ini adalah sub-kelas SkillCard jenis intervensi agresif musuh.

Penarikan Paksa Pesaing: Menarik bidak lawan pilihan dari posisinya saat ini dan memindahkannya tepat di depan petak pemain, biasanya dikombinasikan dengan strategi penjebakan ke daerah padat bangunan monopoli.

2.11.8 Kelas DemolitionCard

Kelas ini adalah sub-kelas SkillCard utilitas sabotase murni.

Penghancuran Properti Real Estate: Menginstruksikan Game untuk menghapus paksa status *building level* rumah/hotel pada suatu objek jalan milik musuh, memukul mundur elevasi ekonominya kembali menjadi tanah kosong tanpa ganti rugi.

2.12. Kelas CardDeck

Kelas ini adalah struktur data tumpukan generik (*template class*) untuk simulasi dek kartu.

Mekanisme Pemisahan Dek & Pembuangan: Memisahkan *state* dua penyimpanan utama; Laci tumpukan ambil (drawPile_) dan tumpukan tempat sampah (discardPile_)

.

Sistem Reshuffle Abadi: Mengaplikasikan keamanan pasokan tak terbatas. Ketika draw() mendapati tumpukan kosong, struktur ini mandiri memindahkan *discard pile* dan memanggil algoritma C++ std::shuffle agar siklus pengambilan kartu tidak akan berujung eror.

2.13. Kelas GameCLI

kelas ini adalah kode utama antarmuka terminal (CLI) yang berinteraksi fisik dengan *user*.

Mesin Pemroses Loop Tangguh: Mengandalkan *infinite dispatch loop* untuk mengurai (*parsing*) untaian kata pengguna (seperti "BANGUN L1"), menerjemahkannya agar *case-insensitive*, dan memetakannya (*routing*) ke fungsi kontroler di sistem permainan.

Barikade Validasi Input: Dilengkapi metode semacam promptInt(), yang membungkus masuknya karakter abjad ke fungsi numerik, melindungi memori dari *Exception* fatal agar konsol/aplikasi tidak *crash* mendadak akibat *typo* pengguna.

Pemasok Badan Logika Lambda: Mengisi kekosongan *callback* anonim kelas inti untuk menerjemahkan pertanyaan spesifik seperti prosedur lelang kompleks dan *prompt* persetujuan Y/N.

2.14. Kelas GamePrinter

Kelas ini adalah spesialis representasi rupa seni berbasis teks (*ASCII-Art Renderer*).

Pelokalisiran Spaghetti Print: Mengisolasi logika panjang *cout* di konsol. Menerjemahkan koordinat memori Board menjadi denah visual matriks kotak-kotak spasial (CETAK_PAPAN) serta merekap tabel properti dengan kolom yang rapi, menjaga kelas antarmuka pusat tetap bersih dari instruksi tipografi.

2.15. Kelas SaveManager

Kelas ini adalah sebaga pengatur berkas permanen (Save/Load State Dunia).

Protokol Serialisasi Berbasis Teks: Tidak menggunakan basis data besar (seperti format XML/JSON murni), melainkan membongkar memori *runtime* telanjang menjadi deretan teks datar (Raw Plain-Text) secara baris per baris.

Re-Hidrasi Jaringan Pointer Memori: Menyelesaikan problematika putusnya untaian *Pointer* C++ setelah program dimatikan. Saat muat ulang (*Load*), sistem menggunakan relasi *Identifier Key* (seperti *username* akun) untuk merajut dan merelasikan ulang objek kepemilikan tanah secara akurat kembali ke *Pointer* memori pemain yang baru hidup, mengamankan seluruh integritas referensi antar-kelas.

2.16. Kelas Config Loader

Kelas ini adalah jembatan *parser* pembuka berkas penyetelan luar sebelum permainan dirender.

Ekstraksi Pengaturan Dinamis: Beroperasi pada fase inisialisasi awal. Mampu mendeteksi *path/directory* dokumen pengaturan (untuk biaya lahan pajak atau parameter *board*) lalu memindahkannya menjadi objek kelas konfigurasi siap saji.

2.17. Kelas LogSerializer

Kelas ini adalah unit *converter* spesifik yang mendukung pergerakan data riwayat aktivitas.

Konverter Bilateral Log: Mengubah untaian rekaman dalam kapsul aksi (misal "Pemain pindah ke A") menjadi *string* padat dan sebaliknya memecah (men-*parsing*) kumpulan teks tunggal menjadi struct murni, krusial bagi TransactionLogger saat menyimpan ke *file* lokal.

2.18. Kelas Data Pembantu (Structs, Value Objects)

Komponen ini adalah kumpulan data pasif yang menjaga kebersihan aliran variabel tanpa perlu diberikan perilaku *method* berlebih.

GameCallbacks: Wadah penyimpanan *std::function lambda*, penghubung sinyal abstrak lapisan permainan langsung melompat menembus UI antarmuka layar.

SpecialConfig & TaxConfig: Entitas kecil layaknya *Singleton* yang berfungsi mendikte konstanta keuangan pakem (denda absolut, batas persentase, pajak tanah final) tanpa dicampuradukkan ke dalam logika matematika properti.

LogEntry: Kapsul *record* murni. Menyimpan siapa pelaku, apa aksi yang dikerjakan, dan ke mana tujuan objek transaksinya secara temporer untuk dikoleksi entitas pencatat raksasa.

3. Justifikasi Desain

3.1. Desentralisasi God Class ke Delegation Model (Pada Game dan Player)

Alasan Pemilihan: Awalnya, kelas Game melakukan segalanya: mengendalikan uang, mengatur pergerakan, managemen property, dan bahkan melaksanakan seluruh putaran pelelangan (God Class Anti-Pattern). Model didelegasikan ke objek yang relevan.

Kelebihan:

- Logika sangat modular. Jika aturan bergerak berubah, perubahannya hanya dilakukan di Player.cpp, tidak menyentuh Game.
- Kemudahan pelacakan bug (isolasi masalah lebih cepat).
- Mengurangi beban memori instansiasi statis pada Game.

Kekurangan:

Meningkatkan penjalaran metode (method chaining). Game masih harus menyediakan fungsi cmdRollDice kosong yang isinya sekadar memanggil player.rollDice(*this). Hal ini membuat jejak kode sedikit melingkar. Mengingat refactor dilakukan dengan waktu yang sangat kecil diakibatkan kesalahan kami, menurut kami ini adalah konsekuensi dari proses pengerjaan kami sendiri.

3.2. Pola Callback/Observer antara Logika (Core) dan Antarmuka (UI)

Alasan Pemilihan: Paket core dilarang keras untuk mencetak output ke console (cout) atau menunggu masukan pengguna (cin). Menggunakan GameCallbacks (berisi pointer fungsi/lambda), paket core dapat meminta UI untuk melakukan prompt (seperti memilih target pada lelang) tanpa perlu mengimpor file UI.

Kelebihan:

Keterlepasan yang ketat (Strict Decoupling). Arsitektur Nimonspoli saat ini bisa dipasangkan dengan program visualisasi antarmuka grafis (seperti OpenGL atau Qt) besok hari tanpa perlu mengubah satu baris kode pun di paket core.

Desain sangat scalable.

Kekurangan:

Kerumitan alur pemanggilan fungsi. Seringkali sulit untuk langsung membaca jalannya kode karena program akan melompat dari core ke fungsi anonim (lambda) di GameCLI lalu kembali lagi ke core.

3.3. Abstraksi dan Polimorfisme pada Petak (Tile) dan Kartu (Card)

Alasan Pemilihan: Sistem Monopoli sangat rentan terhadap spaghetti code jika kita menggunakan rantai kondisional yang besar (misalnya: if(tile == "PAJAK") doA(); else if (tile == "PROPERTI") doB();). Dengan polimorfisme, game hanya perlu memanggil tile->onLanded().

Kelebihan:

Open-Closed Principle: Sistem tertutup untuk modifikasi namun sangat terbuka untuk ekstensi. Menambah jenis petak atau efek kartu baru di masa depan hanya membutuhkan pembuatan kelas turunan baru tanpa mengganggu kode lama.

Kekurangan:

Adanya overhead akibat vtable (metode virtual) di bahasa C++.

Mewajibkan memory management dinamis melalui pemanfaatan pointer (Smart Pointers), yang terkadang menuntut tingkat ketelitian lebih saat objek dipindahkan atau dikloning.

3.4. Penggunaan Log Statis (TransactionLogger)

Alasan Pemilihan: Sistem memerlukan rekam jejak aktivitas, namun mengirimkan logger sebagai parameter di setiap pemanggilan fungsi (seperti bergerak, membeli, denda) merupakan pengotoran dari bentuk fungsi itu sendiri. Menjadikannya metode dan penyimpanan statis memecahkan masalah injeksi dependency.

Kelebihan: Sangat ringkas, praktis, dan dapat dijangkau dari sudut terdalam arsitektur sekalipun (misalnya di dalam method instan Tile).

Kekurangan: Secara multithreading ini tidak aman, dan objek ini akan bertahan selama program berjalan (rentan pada pelacakan status memori global jika diubah secara ceroboh).

3.5. Plain Text Serialization untuk Save/Load (SaveManager)

Alasan Pemilihan: Tidak menggunakan standar seralisasi pihak ketiga (seperti format JSON atau YAML dari pustaka luar) demi meminimalisasi ukuran biner dan menjunjung nilai pembuatan pustaka dari nol.

Kelebihan: Menjamin ketiadaan ketergantungan paket (zero-dependencies). Kecepatan komputasi ekstraksi bit murni sangat cepat.

Kekurangan: File save menjadi format khusus yang rapuh. Jika suatu saat format/urutan angka penyimpanan direstrukturisasi (misalnya menambah field umur_player), maka seluruh file save lama dari permainan akan rusak/tidak terbaca tanpa menyediakan program konverter mundur.

3.6. Kendala yang Dialami dalam Pemilihan Desain OOP

Dalam proses perancangan dan pengembangan aplikasi Nimonspoli ini, kami menemui beberapa kendala signifikan terkait implementasi desain Object-Oriented Programming (OOP):

Game menjadi god class (Oh No): Terjadinya Anti-Pattern God Class pada Tahap Awal Pada rancangan awal permainan, kami secara tidak sengaja memusatkan hampir seluruh fungsionalitas dan alur logika ke dalam satu kelas utama, yaitu kelas Game. Kelas ini mengendalikan segalanya: mulai dari logika pelemparan dadu pemain, perpindahan, penagihan uang transaksi, semua hal terkait properti dan cards, hingga menampung perulangan (loop) pelelangan. Hal ini menyebabkan kelas Game menjelma menjadi sebuah God Class yang sangat membengkak (bloated). Kondisi ini melanggar prinsip tanggung jawab tunggal (Single Responsibility Principle), sehingga kode menjadi sulit untuk dibaca, dilacak bug-nya, dan di-maintain. (Kendala ini pada akhirnya memaksa kami melakukan proses refactoring ekstensif di tengah jalan untuk mendistribusikan logika ke kelas Player, TransactionLogger, dan AuctionManager).

Strong Dependencies: Kesulitan Implementasi Akibat Ketergantungan (Coupling) yang Tinggi Antar Kelas Dampak dari pemisahan kelas-kelas OOP yang begitu detail (seperti memisahkan Player, Board, Tile, Property, dan kumpulan Card) adalah munculnya jaring ketergantungan antar kelas yang sangat rumit. Sebagai contoh, pergerakan pemain membutuhkan interaksi dengan Board; saat mendarat di Tile, petak tersebut butuh mengubah status internal Player sekaligus memicu efek global di Game. Tingginya referensi silang (cross-referencing) ini sering kali memicu konflik kompilasi seperti circular dependency dan membuat proses perangkaian (wiring) antar kelas di tahap implementasi menjadi sangat menyulitkan.

Long Initialization: Proses Inisialisasi Arsitektur yang Sangat Memakan Waktu Tahap awal pembuatan struktur kode (sebelum masuk ke penulisan logika permainan yang sesungguhnya) memakan porsi waktu yang sangat panjang. Hal ini diakibatkan oleh banyaknya aturan dan semantic design dari paradigma OOP modern (khususnya C++) yang harus dipatuhi dengan ketat. Kami harus mendesain hierarki pewarisan virtual yang tepat, mengatur manajemen memori secara ketat (seperti menggunakan std::unique_ptr pada jajaran Tile), menyusun sistem template pada struktur data CardDeck, serta membangun mekanisme jembatan Callback (Lambda Function) agar sistem inti permainan tidak bergantung secara kotor (hardcoded) pada fungsi input/output di console. Kepatuhan pada standar arsitektur semantik inilah yang membuat proses pengerjaan rangka aplikasi memakan waktu jauh di atas perkiraan awal.

4. Penerapan Konsep OOP

4.1. Inheritance & Polymorphism

Inheritance dan polymorphism diterapkan secara konsisten pada hierarki utama permainan untuk memastikan *Open/Closed Principle*, di mana entitas baru dapat ditambahkan tanpa memodifikasi kode pemanggil (seperti kelas Game atau Board).

Hierarki Tile

Tile adalah kelas abstrak dengan *pure virtual method* onLanded(). Terdapat sembilan kelas konkret yang mewarisinya: PropertyTile, GoTile, JailTile, FreeParkingTile, GoToJailTile, TaxTile, FestivalTile, ChanceTile, dan CommunityTile. Kelas Board menyimpan elemen papan menggunakan vector<unique_ptr<Tile>> dan memanggil onLanded() secara polimorfik tanpa perlu mengetahui jenis konkret petaknya.

```
// include/core/tiles.h
class Tile {
public:
    virtual void onLanded(Player& player, Game& game) = 0;
    virtual ~Tile() = default;
};

class GoToJailTile : public Tile {
public:
    void onLanded(Player& player, Game& game) override {
        game.sendToJail(player);  // Polimorfisme: efek berbeda per jenis Tile
    }
};

class FreeParkingTile : public Tile {
public:
    void onLanded(Player&, Game&) override {}  // Tidak ada efek
};
```

Keuntungan: Board dapat menyimpan seluruh 40 petak dalam satu container homogen dan memproses pendaratan pemain dengan satu baris kode: board_->getTile(pos)->onLanded(player, *this). Penambahan jenis petak baru tidak memerlukan perubahan pada Board atau Game.

Hierarki Property

Perubahan besar dari desain sebelumnya adalah pendelegasian fungsionalitas ke kelas Property. Property kini adalah kelas abstrak dengan metode virtual untuk kalkulasi sewa (calcRent) dan penanganan aksi (*delegated logic*) seperti handlePurchase, handleRent, handleAuction, dan performLiquidation. Tiga kelas konkret mewarisinya: Street, Railroad, dan Utility.

```
// include/core/property.h
class Property {
Public:
      virtual int calcRent(int diceTotal = 0) const = 0;
```

```
      // Delegated logic dari Game
      virtual void handlePurchase(Player& player, Game& game);
      virtual void handleRent(Player& payer, int diceTotal, Game& game);
      virtual void performLiquidation(Player& player, Game& game);
      // ...
};
// include/core/propertytypes.h
class Railroad : public Property {
Public:
      int calcRent(int diceTotal = 0) const override {
      auto it = cfg_.rentTable.find(ownerRRCount_);
      return (it != cfg_.rentTable.end()) ? it->second : 0;
}
};
```

Keuntungan: Logika yang sangat berbeda untuk aset bertipe Street, Railroad, atau Utility (seperti Railroad yang menghitung sewa dari jumlah stasiun yang dimiliki) sepenuhnya terlokalisasi di masing-masing *subclass*. Game cukup memanggil prop.handlePurchase().

Hierarki Card

Card adalah kelas abstrak dengan *pure virtual method* execute(). Kelas ini diwarisi oleh ChanceCard, CommunityCard, dan SkillCard. SkillCard menambahkan lapisan abstraksi kedua dengan *pure virtual method* use(). Enam turunan konkret mewarisi SkillCard: MoveCard, DiscountCard, ShieldCard, TeleportCard, LassoCard, dan DemolitionCard. Sementara itu, ChanceCard diwarisi oleh tiga kelas (seperti ChanceGoNearestStation) dan CommunityCard diwarisi oleh tiga kelas (seperti CommunityBirthday).

```
// include/core/cards.h
class Card {
public:
    virtual void execute(Player& player, Game& game) = 0;
};
```

```
class SkillCard : public Card {
public:
    virtual void use(Player& player, Game& game) = 0;
    void execute(Player& p, Game& g) override { use(p, g); }
};

class ShieldCard : public SkillCard {
public:
    void use(Player& player, Game&) override {
        player.setShielded(true);  // Efek unik ShieldCard
    }
};
```

Keuntungan: Kontainer CardDeck<SkillCard> dapat menyimpan semua jenis SkillCard secara homogen. Sistem dapat menampilkan deskripsi dan mengeksekusi kartu apapun tanpa harus melakukan *branching* per jenis kartu.

4.2. Method/Operator Overloading

Operator overloading diterapkan pada kelas Player untuk dua tujuan: manipulasi balance dan perbandingan kekayaan (net worth) .

Operator += dan -= untuk Manipulasi Balance

```
// include/core/player.h
Player& operator+=(int amount) {
    balance_ += amount;
    return *this;
}
Player& operator-=(int amount) {
    balance_ -= amount;
    return *this;
```

```
}

// Penggunaan di Bank (include/core/bank.h):
void pay(Player& player, int amount)  { player += amount; }
void collect(Player& player, int amount) { player -= amount; }
void transfer(Player& from, Player& to, int amount) {
    from -= amount;
    to   += amount;
}
```

Keuntungan: kode di Bank, Game, dan Card menjadi lebih ekspresif dan mudah dibaca. Seluruh modifikasi balance terpusat melalui operator yang terdefinisi, sehingga lebih mudah diintersepsi untuk logging jika diperlukan di masa mendatang.

Operator < dan > untuk Perbandingan Net Worth

```
// include/core/player.h
bool operator<(const Player& other) const {
    return this->netWorth() < other.netWorth();
}
bool operator>(const Player& other) const { return other < *this; }

// Penggunaan di Game::winners() (src/core/game.cpp):
sort(active.begin(), active.end(),
    [](Player* a, Player* b){ return *b < *a; });
```

Keuntungan: urutan pemain berdasarkan net worth pada kondisi akhir giliran dapat dilakukan langsung dengan std::sort dan lambda yang memanfaatkan operator yang telah di-overload. netWorth() menghitung total uang tunai ditambah nilai beli seluruh properti dan bangunan.

4.3. Template & Generic Classes

Sistem tumpukan kartu dikelola oleh *generic class* CardDeck<T>. CardDeck<T> diinstansiasi menjadi CardDeck<ChanceCard>, CardDeck<CommunityCard>, dan CardDeck<SkillCard> di dalam kelas Game.

```
// include/core/carddeck.h
template <typename T>
class CardDeck {
public:
    T* draw() {
        if (drawPile_.empty()) reshuffleDiscard();
        if (drawPile_.empty()) throw runtime_error("CardDeck is empty.");
        // ...
        return inFlightPile_.back().get();
    }
    void shuffle();
    void discard(T* card);
    // ...
};
```

CardDeck<T> diinstansiasi tiga kali dalam Game:

```
// include/core/game.h
CardDeck<ChanceCard>   chanceDeck_;
CardDeck<CommunityCard> communityDeck_;
CardDeck<SkillCard>    skillDeck_;
```

Keuntungan: mekanisme deck (draw, discard, reshuffle) identik untuk ketiga jenis kartu. Tanpa template, tiga kelas deck terpisah dengan isi yang hampir identik harus dibuat dan dipelihara secara independen. Perubahan pada logika deck (misalnya cara reshuffle atau penanganan deck kosong) cukup dilakukan di satu tempat.

4.4. Exception

Penanganan galat (*Exception Handling*) diterapkan untuk memvalidasi aksi pada kelas Core, dan ditangkap (*catch*) oleh lapisan UI.

Kelas-kelas yang mengimplementasikan *Exception*:

- **Dice::setRoll()**: Melempar std::invalid_argument jika input dadu di luar batas 1–6.
- **Street::addBuilding()**: Melempar std::logic_error jika mencoba membangun di petak yang sudah mencapai level hotel.
- **Player::removeFromHand()**: Melempar std::out_of_range jika indeks kartu di luar batasan ukuran *hand* pemain.
- **CardDeck::draw()**: Melempar std::runtime_error jika mencoba menarik dari *deck* yang sudah benar-benar kosong tak tersisa.

Validasi di Core Layer

```
// src/core/game.cpp - cmdBuild()
void Game::cmdBuild(const string& code) {
    Property* prop = board_->getProperty(code);
    if (!prop || prop->type() != PropertyType::STREET)
        throw invalid_argument("Kode properti tidak valid.");
    if (prop->owner() != &player)
        throw invalid_argument("Properti bukan milikmu.");
    if (!street->hasMonopoly())
        throw logic_error("Kamu belum memonopoli color group ini.");
    if (street->hasHotel())
        throw logic_error("Properti sudah memiliki hotel.");
    if (!player.canAfford(cost))
        throw runtime_error("Uang tidak cukup untuk membangun.");
    // ... lanjut membangun
```

```
}
```

Penangkapan di UI Layer

```
// src/ui/gamecli.cpp - dispatch()
try {
    dispatch(line);
} catch (const exception& e) {
    printError(e.what());
}

// src/ui/gamecli.cpp - cmdBangun()
try {
    game_.cmdBuild(target->code());
    cout << "Berhasil membangun...\n";
} catch (const exception& e) {
    printError(e.what());
}

// include/core/dice.h
pair<int,int> setRoll(int d1, int d2) {
   if (d1 < 1 || d1 > 6 || d2 < 1 || d2 > 6)
          throw std::invalid_argument("Dice values must be between 1 and 6.");
          // ...
          }
```

Keuntungan: Lapisan logika tidak perlu menangani implementasi terminal (cout/cin) untuk memberi pesan kesalahan; cukup memberitahukan UI untuk menampilkannya lewat antarmuka yang ada.

4.5. C++ Standard Template Library

STL digunakan secara pervasif di seluruh lapisan. Penggunaan yang krusial terhadap struktur data dan mekanisme permainan:

Kontainer/Algoritma	Lokasi Penggunaan	Alasan Pemilihan
vector<unique_ptr<Tile>>	Board::tiles_	Akses O(1) by index; urutan petak yang fix sepanjang permainan
map<string, unique_ptr<Property>>	Board::properties_	Lookup O(log n) by kode properti; kode dijamin unik
vector<Property*>	Player::properties_	Agregasi referensi non-owning; size dinamis sesuai kepemilikan
vector<SkillCard*>	Player::hand_	Kartu tangan maksimal 3; iterasi dan hapus O(n) dapat diterima
map<int,int>	RailroadConfig, UtilityConfig	Lookup sewa/multiplier by jumlah properti yang dimiliki
array<int,6>	Street::rentTable_	Tabel sewa fixed 6 level; akses O(1) by indeks level
std::shuffle	CardDeck::reshuffleDiscard()	Pengacakan kartu deck yang benar secara statistik
std::sort	Game::winners()	Pengurutan pemain by net worth untuk penentuan pemenang
std::mt19937	Dice, CardDeck	RNG berkualitas tinggi untuk simulasi dadu dan pengacakan kartu

Contoh penggunaan map dalam penentuan sewa Railroad:

```
// include/core/config.h
class RailroadConfig {
public:
    std::map<int,int> rentTable;  // jumlah RR -> biaya sewa
};

// include/core/propertytypes.h
int Railroad::calcRent(int) const {
    auto it = cfg_.rentTable.find(ownerRRCount_);
    return (it != cfg_.rentTable.end()) ? it->second : 0;
}
```

Contoh penggunaan vector dan std::sort pada penentuan pemenang:

```
// src/core/game.cpp - Game::winners()
vector<Player*> active = activePlayers();
sort(active.begin(), active.end(),
    [](Player* a, Player* b){ return *b < *a; });
// Tie-break 1: net worth
int topWorth = active[0]->netWorth();
vector<Player*> tied;
for (auto* p : active)
    if (p->netWorth() == topWorth) tied.push_back(p);
// Tie-break 2: jumlah properti, lalu jumlah kartu
```

4.6. Konsep OOP lain

4.6.1 Komposisi (Composition)

Komposisi adalah relasi has-a di mana lifetime objek yang dimiliki sepenuhnya dikendalikan oleh objek pemilik. Dalam Nimonspoli, Game mengomposisi Board, Bank, Dice, dan TransactionLogger baik sebagai nilai langsung maupun via unique_ptr. Ketika Game dihancurkan, seluruh objek komposisi ikut dihancurkan secara otomatis tanpa destruktor manual.

```
// include/core/game.h
class Game {
private:
    unique_ptr<Board>             board_;   // Komposisi via unique_ptr
    vector<unique_ptr<Player>>    players_; // Game memiliki semua Player
    Dice                          dice_;   // Komposisi sebagai nilai
    Bank                          bank_;
    TransactionLogger             logger_;
    CardDeck<ChanceCard>          chanceDeck_;
    CardDeck<CommunityCard>       communityDeck_;
    CardDeck<SkillCard>           skillDeck_;
};
```

Board juga mengomposisi Tile dan Property:

```
// include/core/board.h
class Board {
private:
    vector<unique_ptr<Tile>>         tiles_;      // Board memiliki Tile
    map<string, unique_ptr<Property>>  properties_; // Board memiliki Property
};
```

Keuntungan: kepemilikan resource menjadi eksplisit dan tidak ambigu. Tidak ada kebocoran memori karena unique_ptr memanggil destruktor secara otomatis saat keluar dari scope. Semantik kepemilikan yang jelas juga mencegah double-free dan dangling pointer.

4.6.2 Agregasi (Aggregation)

Agregasi adalah relasi has-a yang lebih lemah, di mana objek yang diacu tetap hidup secara independen. Player menyimpan daftar Property* dan SkillCard* sebagai raw pointer (bukan unique_ptr) karena Player tidak memiliki lifetime dari objek-objek tersebut; Board dan CardDeck lah yang menjadi pemilik sesungguhnya.

```
// include/core/player.h
class Player {
private:
    vector<Property*>  properties_;  // Agregasi: referensi non-owning
    vector<SkillCard*> hand_;        // Agregasi: referensi non-owning
};
```

Keputusan ini krusial untuk mekanisme pengambilalihan aset saat kebangkrutan. Saat properti berpindah dari debtor ke creditor, operasinya sesederhana mengubah pointer dan memperbarui daftar agregasi masing-masing pemain, tanpa memindahkan kepemilikan resource yang sesungguhnya:

```
// src/core/game.cpp - handleBankruptcy()
for (auto* prop : debtor.properties()) {
    prop->setOwner(creditor);        // Ganti owner di Property
    creditor->addProperty(prop);     // Tambah ke agregasi creditor
```

```
    refreshPropertyCounts(creditor); // Update monopoly/RR/util count
}
// Board tetap sebagai pemilik unique_ptr<Property> — tidak ada move
```

Keuntungan: pemisahan antara "referensi kepemilikan game" (Player::properties_) dan "kepemilikan resource" (Board::properties_) membuat operasi transfer properti menjadi ringan dan aman. Tidak ada risiko double-free atau dangling pointer karena Board selalu hidup lebih lama dari Player.

4.6.3 Enkapsulasi (Encapsulation)

Enkapsulasi diterapkan secara konsisten dengan menjadikan seluruh atribut kelas sebagai private, dan menyediakan akses terkontrol melalui getter/setter. Atribut yang bersifat read-only hanya memiliki getter. Atribut yang dapat dimodifikasi memiliki setter dengan nama deskriptif yang mengkomunikasikan maksud perubahan:

```
// include/core/player.h - Contoh enkapsulasi Player
class Player {
public:
    int  balance()      const { return balance_; } // read-only
    bool canAfford(int amount) const { return balance_ >= amount; }
    void setStatus(PlayerStatus s)   { status_ = s; }
    void setShielded(bool v)         { shielded_ = v; }
    void resetTurnState() {          // Satu method untuk semua reset
        usedCardThisTurn_ = false;
        hasRolled_      = false;
        shielded_       = false;
        clearDiscount();
    }
private:
    int          balance_     = 0;
```

```
    PlayerStatus status_             = PlayerStatus::ACTIVE;
    bool         shielded_     = false;
    bool         usedCardThisTurn_= false;
    bool         hasRolled_    = false;
};
```

Keuntungan: invariant kelas terjaga secara struktural. Misalnya, balance tidak dapat dimodifikasi secara sembarangan dari luar; semua perubahan harus melalui operator+= dan operator-= yang terdefinisi, atau melalui Bank yang merupakan satu-satunya fasad transaksi yang sah.

4.6.4 Pola Desain Callback / Strategy (melalui GameCallbacks)

GameCallbacks adalah struct yang menyimpan kumpulan std::function sebagai "slot" untuk logika UI yang perlu dipanggil oleh Core Layer. Pola ini merupakan adaptasi dari Strategy Pattern: Core Layer mendefinisikan kapan aksi perlu terjadi, sementara UI Layer mendefinisikan bagaimana aksi tersebut dilakukan.

```
// include/core/game.h
class GameCallbacks {
public:
    // Slot untuk keputusan interaktif yang memerlukan UI
    function<bool(Property&)>              onOfferPurchase;
    function<void(Player&)>                onTaxPPH;
    function<void(Player&)>                onFestival;
    function<void(Player&,int,Player*)>   onLiquidation;
    function<int(Player&)>                 onTeleport;
    function<Player*(Player&)>             onLasso;
    function<string(Player&)>              onDemolition;
    function<void(Player&)>                onDropCard;
};
```

```
// src/ui/gamecli.cpp - Registrasi callback oleh GameCLI
cb.onOfferPurchase = [this](Property& prop) -> bool {
    return promptBuyStreet(prop);  // Implementasi CLI
};
game_.setCallbacks(move(cb));
```

Keuntungan: Core Layer sepenuhnya bebas dari ketergantungan pada kelas UI konkret. Mengganti implementasi CLI dengan GUI hanya memerlukan registrasi callback baru tanpa menyentuh satu baris pun kode di Game, Board, atau Player. Pengujian unit Core Layer dapat dilakukan dengan menyediakan callback mock.

4.6.5 Layered Architecture

Program mengimplementasikan tiga-lapis arsitektur yang memisahkan tanggung jawab secara tegas:

Lapisan	Direktori	Kelas Utama	Tanggung Jawab
User Interaction Layer	include/ui/, src/ui/	GameCLI, BoardPrinter	Input/output terminal, parsing perintah, prompt interaktif, rendering papan berwarna
Game Logic Layer	include/core/, src/core/	Game, Board, Player, Tile, Card, Property, Dice, Bank, TransactionLogger	Aturan permainan, kalkulasi sewa, manajemen state, distribusi kartu
Data Access Layer	include/data/, src/data/	ConfigLoader, SaveManager	Pembacaan file konfigurasi, serialisasi dan deserialisasi state permainan

Aturan ketergantungan antar layer: UI Layer boleh mengakses Core Layer dan Data Layer; Core Layer tidak boleh mengakses UI Layer (diimplementasikan melalui GameCallbacks); Data Layer mengakses Core Layer untuk membaca/menulis state. Aturan ini mencegah coupling sirkular dan memastikan setiap layer dapat dikembangkan atau diganti secara independen.

4.6.6 Prinsip Single Responsibility (dari SOLID)

Setiap kelas dirancang untuk memiliki satu alasan untuk berubah:

- Bank: satu-satunya kelas yang memodifikasi balance Player. Jika aturan transaksi berubah (misalnya penambahan pajak transaksi), hanya Bank yang perlu diubah.
- TransactionLogger: satu-satunya kelas yang menyimpan riwayat aksi. Perubahan format log hanya berdampak pada kelas ini.
- ConfigLoader: satu-satunya kelas yang membaca file konfigurasi. Perubahan format file konfigurasi hanya berdampak di sini.
- BoardPrinter: satu-satunya kelas yang menangani rendering visual. Perubahan tampilan papan (misalnya dari ANSI ke GUI) terisolasi di kelas ini.
- FestivalEffect: satu-satunya kelas yang mengelola logika penggandaan sewa festival. Aturan festival yang berubah hanya berdampak di FestivalEffect, bukan di Street secara keseluruhan.

4.6.7 Prinsip Liskov Substitution (dari SOLID)

Seluruh subclass Tile, Property, dan Card dapat menggantikan superclass-nya tanpa merusak fungsionalitas program. Contoh konkret: Board menyimpan vector<unique_ptr<Tile>> dan memanggil onLanded() pada setiap elemen. Board tidak perlu mengetahui apakah tile yang sedang diproses adalah GoTile, PropertyTile, atau FestivalTile; seluruhnya berperilaku benar sesuai kontrak yang didefinisikan Tile. Hal yang sama berlaku untuk Game yang memanggil prop.calcRent() tanpa mengetahui jenis Property konkret.

Implementasi turunan tidak memperlemah precondition atau memperkuat postcondition dibandingkan superclass. Misalnya, seluruh override onLanded() menerima Player& dan Game& sebagai argumen (precondition sama) dan tidak mengembalikan nilai (postcondition sama). Tidak ada subclass yang menambahkan throw-specification baru yang bertentangan dengan kontrak Tile.

5. Bonus

5.1. Kreasi Mandiri

5.1.1. Callback Architecture untuk Tampilan UI

Kami menghindari pencampuran logika permainan dengan input/output (menghindari pemanggilan hardcode std::cin atau std::cout di dalam struktur core/). Kelas Game menggunakan functions GameCallbacks untuk memicu prompt ke pengguna. Kelas GameCLI di lapisan antarmuka bertugas menyuntikkan implementasi fungsional ke dalam callback ini. Hal ini membuat sistem murni terdesentralisasi (Decoupled). Jika di masa depan antarmuka permainan ingin diganti dari Command Line menjadi UI Grafis interaktif, modifikasi hanya perlu membuang GameCLI dan menempelkan UI baru tanpa menyentuh sebaris pun algoritma di lapisan inti. Namun memang dari kami sendiri tidak memiliki waktu yang memadai untuk mengimplementasi UI diluar dari CLI.

Cuplikan Kode Relevan:

```
// include/core/game.h
class GameCallbacks {
public:
    // Slot untuk keputusan interaktif yang memerlukan UI
    function<bool(Property&)>              onOfferPurchase;
    function<void(Player&)>                onTaxPPH;
    function<void(Player&)>                onFestival;
    function<void(Player&,int,Player*)>   onLiquidation;
    function<int(Player&)>                 onTeleport;
    function<Player*(Player&)>             onLasso;
    function<string(Player&)>              onDemolition;
    function<void(Player&)>                onDropCard;
};
```

6. Pembagian Tugas

Modul (dalam poin spek)	Implementer	Tester
Pembacaan File Konfigurasi	13524021	13524021, 13524085, 13622076
Save & Load State	13524021, 13622076	13622076, 13524021
Transaction Logger	13524021	13524009, 13524107, 13524021
Dadu & Giliran	13524107	13524009, 13622076, 13524107, 13524021
Petak Spesial	13524107, 13622076	13524009, 13622076, 13524107, 13524021
Sistem Penjara	13524107	13524009, 13622076, 13524107, 13524021
Kepemilikan & Beli Properti	13524009	13524009, 13524021, 13524085
Mekanisme Sewa	13524009	13524009, 13524021, 13524085
Gadai dan Tebus	13524009	13524009, 13524021, 13524085
Bangun	13524107, 13524085	13524009, 13524021, 13524085, 13524107
Lelang	13524009, 13524085	13524009, 13622076, 13524021
Pajak	13524009	13524009, 13524021,13524085
Festival	13524021, 13524085	13524107, 13524021, 13524085

Kebangkrutan & Likuidasi	13524021	13524107, 13524021, 13524085
Kondisi Menang	13524021	13524009, 13524021, 13524085
Kesempatan & Dana Umum	13524021	13524107, 13524021, 13622076
Kartu Kemampuan	13524107, 13524085	13524009, 13622076, 13524021
Cetak Papan	13622076	13622076, 13524021, 13524085
Cetak Akta & Properti	13524021, 13622076	13524009, 13622076

Lampiran

Lampiran A. Form Asistensi

https://drive.google.com/drive/folders/1FoX0ZcHriVJh96dGs58nvxy1zpSst54y?usp=sharing

https://github.com/Labpro-22/tugas-besar-1-omb.git